...while the command-line parameter of the target qemu migration is:

```
-incoming tcp://0:port
```

Migration support in qemu is currently being overhauled to a newer, simpler version, and more functionality is being added. Support for some previously-supported functionality, like migration via SSH or via a file isn't added yet in the new infrastructure—could be an interesting area for people looking to contribute to explore.

Advantages of the KVM approach

There are several advantages in doing things the Linux way: we reuse all the existing software and infrastructure available, and no new commands need to be learned and not many need to be introduced. For example, kill(1) and top(1) work as expected on the guest task on the host system. The guests are scheduled as regular processes. As we saw in Figure 1, each guest consists of two parts: the userspace part (qemu) and the guest part (the guest itself). The guest physical memory is mapped in the task's virtual memory space, so guests can be swapped as well. Virtual processors in a VM are merely threads in the host process.

When KVM was initially written, the design parameters were to support x86 hosts, focus only on full virtualisation (no modifications to guest OS) and with no modifications to the host kernel. However, as KVM started gaining developers and interesting use-cases, things changed. Because of the simple and elegant solution, developers took a liking to the approach and new architecture ports for s390, PowerPC and IA-64 were added within months of starting the ports. Paravirtualisation support also has been added, and pv drivers for net and block devices are available. If a guest OS can communicate with the host, several activities can be speeded up, like network activity or disk IO. Also, modifications to the host OS (Linux) that improve scheduling and swapping, among other things, were proposed and accepted.

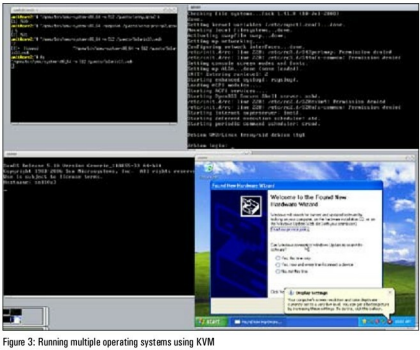

Figure 3: Running multiple operating systems using KVM

KVM seamlessly works across all machine types: servers, desktops, laptops, and even embedded boards. Let's look at each, one by one.

- **Servers:** There's the distinct advantage of being able to use the same management tools and infrastructure as Linux uses. KVM integrates with the Linux scheduler, IO stack, all available filesystems and supports live migration; it has ready support for NUMA and 4096-processor machines.
- **Desktops/laptops:** KVM works on anything Linux works on. The normal desktop doesn't change. You can suspend/resume work as expected, even while virtual machines are running.
- **Embedded:** Linux already supports lots of boards, architectures and machine types. Real-time scheduling is supported. And if you're wondering why one would want to use virtualisation on an embedded machine, here are a few of the many reasons: to sandbox untrusted code, reliable remote kernel upgrades, uniprocessor software on multiprocessor cores, running legacy applications, etc.

All this highlights the differences between KVM and some of the other virtualising solutions.

There's a lot of ongoing work to stabilise KVM and improve the performance of the block and net paravirtualised drivers. There is also some development activity in the regression test suite framework that's ongoing. If you're looking to contribute to KVM, these areas are waiting for you! END

By: Amit Shah

The author has been working on systems software on the Linux kernel for seven years now. He considers himself to be fortunate enough to be accepted as the first off-site developer for the KVM team when he joined Qumranet in 2007. He's worked on a few interesting problems in KVM and is now part of the bigger virtualisation group in the Red Hat family.